BLOCKCHAIN-BASED CERTIFICATE AND DOCUMENT NOTARIZATION SYSTEM

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Use Case Report

submitted by

BOLAGANI LOKESH SAI VENKAT

23501A0517

Under the guidance of

Mr. A. Prashant, Asst. Prof.



Department of Computer Science and Engineering

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

2026-27

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

CERTIFICATE

This is to certify that the Use Case report entitled **"BLOCKCHAIN-BASED CERTIFICATE AND DOCUMENT NOTARIZATION SYSTEM"** that is being submitted by **BOLAGANI LOKESH SAI VENKAT(23501A0517)**, as part of Assignment-2 for the **Blockchain Technology**(**23CS4701B)** course in **4-1** during the academic year **2026-27**.

Course Coordinator
Mr. A. Prashant
Assistant Professor,
Department of CSE,
PVPSIT, Vijayawada

Head of the Department
Dr. A. Jayalakshmi,
Professor and Head,
Department of CSE,
PVPSIT, Vijayawada

MARKS
ASSIGNMENT-2: _____ /5

INDEX

S. No.	Chapter	Page No.
1	Introduction	01
2	Background	02
3	Blockchain Basics	04
4	Use Case Overview	06
5	Implementation	12
6	Benefits	16
7	Challenges	18
8	Conclusion	20
9	SDG’s Addressed	21
10	References	22
11	Appendix A	23

1. INTRODUCTION

1.1 Overview of Blockchain Technology

Blockchain technology is a distributed ledger technology that records transactions in a sequence of cryptographically linked blocks. It allows participating systems to maintain a consistent record of transactions without relying entirely on a single centralized authority. Transactions are validated according to the rules of the blockchain network and, after confirmation, become part of its transaction history.

Each block contains information such as transaction data, a timestamp, and cryptographic information linking it to preceding blocks. This structure helps make unauthorized alteration detectable. Smart contracts extend blockchain functionality by allowing predefined rules to be executed when users submit valid transactions.

Blockchain technology is used in financial systems, supply-chain tracking, digital identity, healthcare records, and document verification. Its properties of traceability, tamper evidence, and programmable access control make it useful in applications where multiple parties need to verify shared records.

1.2 Relevance of Blockchain in Document Notarization

Certificates and important documents are commonly issued and verified through paper records, email communication, institutional databases, or online verification portals. These methods can involve delays, manual effort, and difficulties in maintaining reliable records over time.A digital document can also be modified after issuance. A verifier may receive a file that looks similar to the original but contains altered information. Without access to a trusted reference, it can be difficult to determine whether the received file is identical to the registered version.ChainProof addresses this problem by recording the cryptographic hash of a document on a blockchain. The original document remains off-chain, while the blockchain stores a reference that can be used to check its integrity.When a document is submitted for verification, the application calculates its SHA-256 hash and compares it with the hash registered in the smart contract. If the hashes match and the record has not been revoked, the file passes the system's integrity check.This approach provides a transparent and traceable mechanism for document verification without storing complete documents on the blockchain.

1.3 Purpose of ChainProof

The primary purpose of ChainProof is to provide a simple web-based system for registering, verifying, and revoking certificate and document records through blockchain transactions.The prototype demonstrates how cryptographic hashing, smart contracts, a database, and a web application can work together to reduce dependence on manual document verification.

An important distinction is that a matching hash confirms that a file matches the registered bytes. It does not independently prove that the information contained in the document is true. The reliability of the original registration and the authorization of the issuer remain important.

2. BACKGROUND

Traditional document verification systems face several limitations. These limitations create a need for reliable digital mechanisms to establish document integrity and trace registration activity.

2.1 Document Forgery and Unauthorized Modification

Documents may be altered after they are issued. Changes can include names, marks, dates, identification numbers, or other information. A verifier who receives an altered file may not immediately identify the changes if no trusted reference is available.

A cryptographic hash provides a compact fingerprint of the exact file contents. Registering this fingerprint creates a reference against which a later submission can be compared.

2.2 Dependence on Centralized Databases

Traditional verification systems often depend on a central institution or database. If records become unavailable, are improperly modified, or cannot be accessed by a verifier, the verification process may be delayed.

Blockchain records provide a separate transaction history that can be checked independently of the application's ordinary database. However, the blockchain does not automatically eliminate every centralized dependency: the application may still depend on its backend, database, file storage, and blockchain connection.

2.3 Time-Consuming Verification

Manual verification may require contacting the issuing institution, submitting supporting information, and waiting for a response. These steps can increase processing time, particularly when many documents need to be checked.

ChainProof allows a verifier to compare a submitted document against its registered blockchain record through a web interface.

2.4 Lack of Traceability

A conventional file-sharing system may not provide a reliable, independently verifiable history of document registration and revocation.

By recording relevant operations through smart-contract transactions and events, ChainProof can provide transaction references that support auditing.

2.5 Difficulty Detecting Modified Files

A document may appear visually unchanged even when its underlying file bytes differ. For example, saving a PDF again can alter metadata or file structure.

ChainProof compares SHA-256 hashes rather than relying on visual appearance. Even a small change to the file bytes normally produces a different digest.

2.6 Unauthorized Registration

If anyone can register arbitrary documents, an attacker could create a blockchain record for a fraudulent certificate. Therefore, document integrity alone is not sufficient.

ChainProof includes issuer authorization controls. Only an authorized issuer should be permitted to register documents, and administrators can manage issuer authorization according to the contract rules.

This mechanism establishes who is permitted to submit a record; it does not independently prove that every statement in a registered document is genuine.

2.7 Record Revocation

A document may need to be revoked because it was issued incorrectly, replaced, or found to be invalid. An integrity check alone would not detect this if the uploaded file still matched the original hash.

ChainProof therefore includes a revocation mechanism. During verification, the system checks the record's status and must not report a revoked record as valid.

2.8 Need for a Practical Digital Solution

A useful solution must combine integrity verification, controlled registration, status tracking, and a straightforward user interface. It should also avoid unnecessary blockchain storage costs and protect the original documents from unauthorized access.

ChainProof demonstrates these requirements through a local blockchain prototype that can be tested and presented in an academic environment.

3. BLOCKCHAIN BASICS

Blockchain technology provides the underlying mechanism for maintaining the registration and status records used by ChainProof.

3.1 Decentralization

Decentralization refers to distributing ledger maintenance and transaction validation across multiple participating nodes rather than depending on one central database administrator.

The current ChainProof prototype uses a local Hardhat development blockchain. This is suitable for development and demonstration, but it is not a decentralized public production network. Public-network deployment would require a separate deployment and configuration process.

3.2 Immutability and Tamper Evidence

Blockchain transactions, once confirmed, form part of the network's transaction history. Smart-contract state changes are governed by contract rules and are not ordinarily edited like rows in a conventional database.

Cryptographic hashes help identify changes to document contents. If a submitted document produces a different SHA-256 digest from the registered digest, the system reports a hash mismatch.

Blockchain immutability does not guarantee that an original record was truthful when entered. Incorrect information can still be registered by an authorized participant.

3.3 Transparency and Traceability

Blockchain transactions can be inspected using the appropriate network tools. Transaction hashes provide references to submitted transactions, while contract events can record operations such as document registration and revocation.

ChainProof displays transaction and record information needed for the prototype's verification workflow. Access to the original PDF remains separate from public verification metadata.

3.4 Cryptographic Hashing

A cryptographic hash function converts input data of arbitrary length into a fixed-length digest.

ChainProof uses **SHA-256**, which produces a 256-bit digest, commonly represented as a 64-character hexadecimal string.

For example, if a file is represented by bytes (D), its hash is:

[H = \operatorname{SHA256}(D)]

The application calculates the hash from the exact uploaded file bytes. It does not calculate the hash from the filename.

If a file is changed, its resulting digest will normally differ. The digest is not intended to

reconstruct the original document.

3.5 Smart Contracts

Smart contracts are programs deployed on a blockchain. They enforce predefined rules when transactions are submitted.

ChainProof uses a Solidity smart contract named DocumentNotarization.sol. Its responsibilities include issuer authorization, document registration, record retrieval, and revocation.

The contract checks authorization and prevents invalid operations according to its implemented rules. This reduces reliance on frontend-only checks.

3.6 Consensus Mechanisms

A blockchain network uses a consensus protocol to determine which transactions are accepted and how the network's ledger progresses.

The current prototype runs on a local Hardhat development network with chain ID **31337**. This local development environment should not be described as using Polygon's production consensus mechanism.

If ChainProof is later deployed to Polygon Amoy, its network architecture and consensus description must reflect that actual deployment. The present prototype does not require Polygon deployment to demonstrate its core functionality.

3.7 Public-Key Cryptography and Authorization

Blockchain accounts use public addresses and cryptographic keys to authorize transactions. A transaction signed by an account allows the network to associate that operation with the signing address.

ChainProof uses blockchain account addresses in its issuer authorization model. Its local application also uses signed, short-lived role sessions for access to protected application routes.

The local authentication arrangement is intended for development and demonstration, not as a complete production identity-management solution.

3.8 On-Chain and Off-Chain Storage

On-chain data is recorded through blockchain transactions and smart-contract state. Off-chain data is maintained outside the blockchain, such as in a local database or file storage system.

ChainProof keeps the original PDF in private local storage and stores application metadata in SQLite through Prisma. The smart contract maintains the document registration and status information needed for blockchain verification.

This design avoids storing complete documents and unnecessary personal information directly on the blockchain.

4. USE CASE OVERVIEW

4.1 Objectives

The main objective of ChainProof is to develop a blockchain-based application for document integrity verification.

The specific objectives are:

1. To register certificate and document records through a smart contract.
2. To calculate document fingerprints using SHA-256.
3. To restrict registration to authorized issuers.
4. To verify uploaded files against registered hashes.
5. To identify altered documents.
6. To support document revocation.
7. To provide administrators with issuer-management functionality.
8. To generate QR codes linking to document verification pages.
9. To maintain relevant transaction and audit metadata.
10. To provide a user-friendly web interface for registration and verification.
11. To protect original documents from direct public exposure.
12. To demonstrate the complete workflow using a local blockchain.

4.2 Scope of the System

The ChainProof prototype includes the following components:

1. Administrator

The administrator manages issuer authorization and suspension and can perform permitted administrative actions, including revocation where allowed by the smart contract.

2. Issuer

An authorized issuer registers a document by uploading a PDF. The application calculates its hash and submits the corresponding registration transaction.

3. Verifier

A verifier uploads a document or searches by document ID. The system compares the submitted file's hash with the blockchain record and checks revocation status.

4. Web Application

The application provides registration, dashboards, document details, QR generation, and verification results.

5. Smart Contract

The contract enforces authorization and maintains document registration and revocation state.

6. Database and Local Storage

SQLite stores application metadata, while private local storage holds original PDFs. Public verification does not expose the original document.

7. Local Blockchain

Hardhat provides the development blockchain used to deploy the contract and execute transactions during the demonstration.

The prototype is intended for academic evaluation. Production use would require a stronger identity system, deployment to an appropriate network, a security review, operational

monitoring, and suitable storage and backup arrangements.

4.3 System Architecture

The architecture consists of five principal layers.

4.3.1 User Layer

This layer contains the administrator, issuer, and verifier.

- The administrator manages issuer permissions and permitted administrative operations.
- The issuer registers documents and views registration details.
- The verifier checks document integrity and revocation status.

4.3.2 Web Application Layer

The frontend is developed using Next.js, React, TypeScript, and Tailwind CSS.
It provides forms, dashboards, document details, verification results, and QR-code functionality. The interface communicates with backend routes for protected operations and blockchain interaction.

4.3.3 Backend and Application Logic Layer

The backend performs input validation, SHA-256 hashing, application authorization, metadata processing, and coordination of blockchain transactions.
Signed, short-lived role sessions protect application routes. Server-side checks are necessary because frontend visibility alone cannot secure administrative operations.

4.3.4 Blockchain Layer

The Solidity smart contract is deployed on the local Hardhat network.
It maintains issuer authorization and document registration state. Registration and revocation are performed through actual blockchain transactions, and verification reads the relevant contract state.

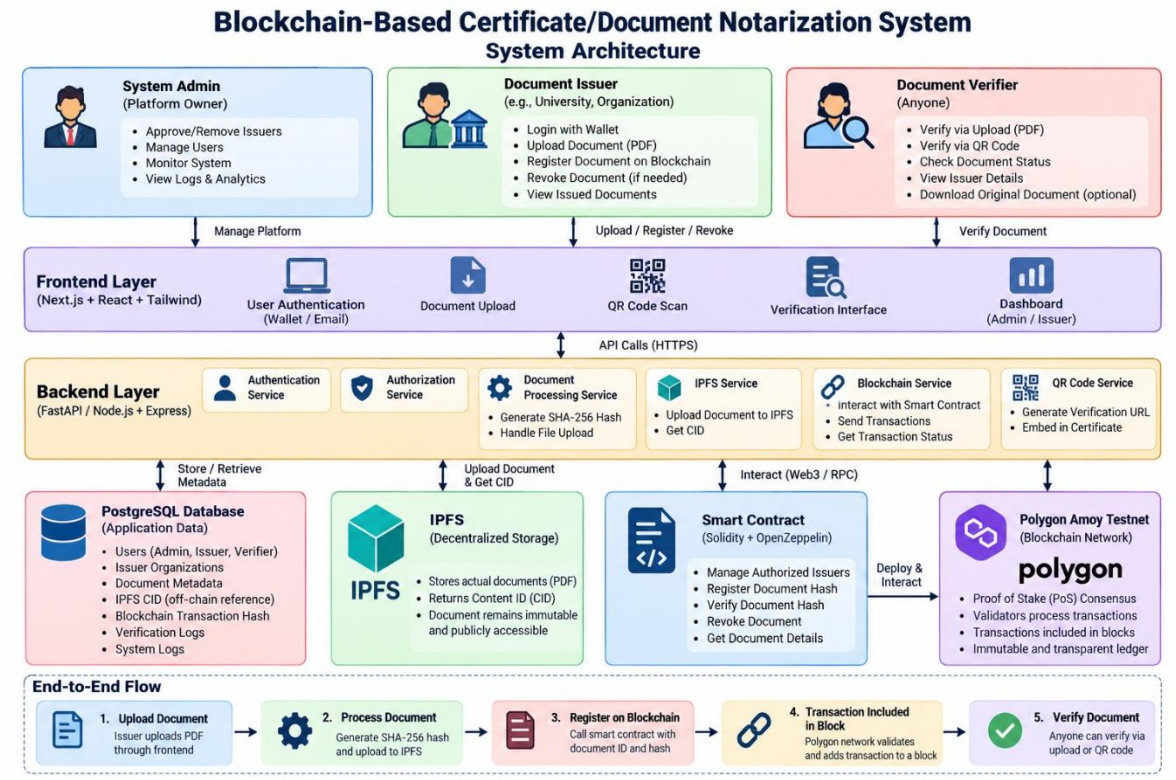
4.3.5 Data Storage Layer

The data layer includes:

- **Smart-contract state:** document identifiers, hashes, issuer information, timestamps, and status information according to the contract implementation.
- **SQLite database:** application metadata and supporting records.
- **Private local file storage:** original PDF files.
- **QR codes:** links to verification pages rather than copies of the documents.

The database and file storage support the application, while the blockchain is used to verify the authoritative registration and revocation state.

Figure 4.1: Architecture of ChainProof
Insert the architecture diagram here.



The backend also interacts with:

- SQLite through Prisma for application metadata.
- Private local storage for original PDFs.
- The QR generator for verification links.

4.4 Actors and Their Responsibilities

Administrator

The administrator authorizes issuers, suspends or reactivates issuers, reviews available records, and performs permitted administrative operations.

Issuer

The issuer uploads a document, provides its descriptive metadata, registers its hash through the smart contract, and obtains a document ID and QR code.

Verifier

The verifier searches for a document or uploads a PDF. The application checks the document hash and revocation status and displays the result.

Blockchain Node

The local Hardhat node processes development transactions and maintains the local blockchain state. It is a development component, not a public production validator network.

4.5 Smart Contracts and Their Functions

The main smart contract is DocumentNotarization.sol.

Its functions support the following operations, according to the implemented contract interface:

- Issuer authorization.
- Issuer suspension and reactivation.
- Document registration.
- Document retrieval.
- Hash-based verification.
- Document revocation.
- Issuer authorization checks.
- Events for relevant state changes.

The administrator's permissions and the issuer's permissions are enforced by contract rules. Invalid or unauthorized operations should revert rather than silently succeed.

4.6 Use of Oracles

An oracle supplies external information to a smart contract when that information cannot be obtained directly from blockchain state.

ChainProof does not require an oracle for its core workflow.

The application can calculate the SHA-256 hash of an uploaded file and compare it with the registered hash obtained from the smart contract. Neither operation requires an external data feed.

An oracle could become relevant in a future extension that depends on external institutional status information or other trusted off-chain facts. Such an extension would require careful consideration of the oracle's reliability.

4.7 Blockchain Type

The current prototype uses a **local development blockchain provided by Hardhat**, with chain ID 31337.

This environment enables contract deployment and transaction testing without requiring public-network funds. It is not a private production blockchain or a Polygon deployment.

The system's smart-contract logic is designed around document registration and verification, while the selected network can be changed in a future deployment if the contract and configuration are compatible.

4.8 Information Recorded in a Block

A blockchain block generally contains a block header and a collection of transactions. Depending on the network, block-level metadata includes information such as a block number, timestamp, parent-block hash, and transaction commitments.

When ChainProof registers a document, a transaction invokes the smart contract. The transaction causes a state change that records the document's integrity information in the contract.

The complete PDF is **not stored in the blockchain block by ChainProof**. The application

retains the original PDF off-chain.

4.9 Document Verification Statuses

The application distinguishes between several results:

- **VALID:** The uploaded file's hash matches the registered hash and the record is not revoked.
- **HASH_MISMATCH:** The document ID exists, but the uploaded file's hash does not match.
- **REVOKED:** The blockchain record is revoked, even if the uploaded file matches the registered hash.
- **NOT_FOUND:** No matching document record exists.
- **VERIFICATION ERROR:** The blockchain connection or another necessary operation failed, preventing reliable verification.

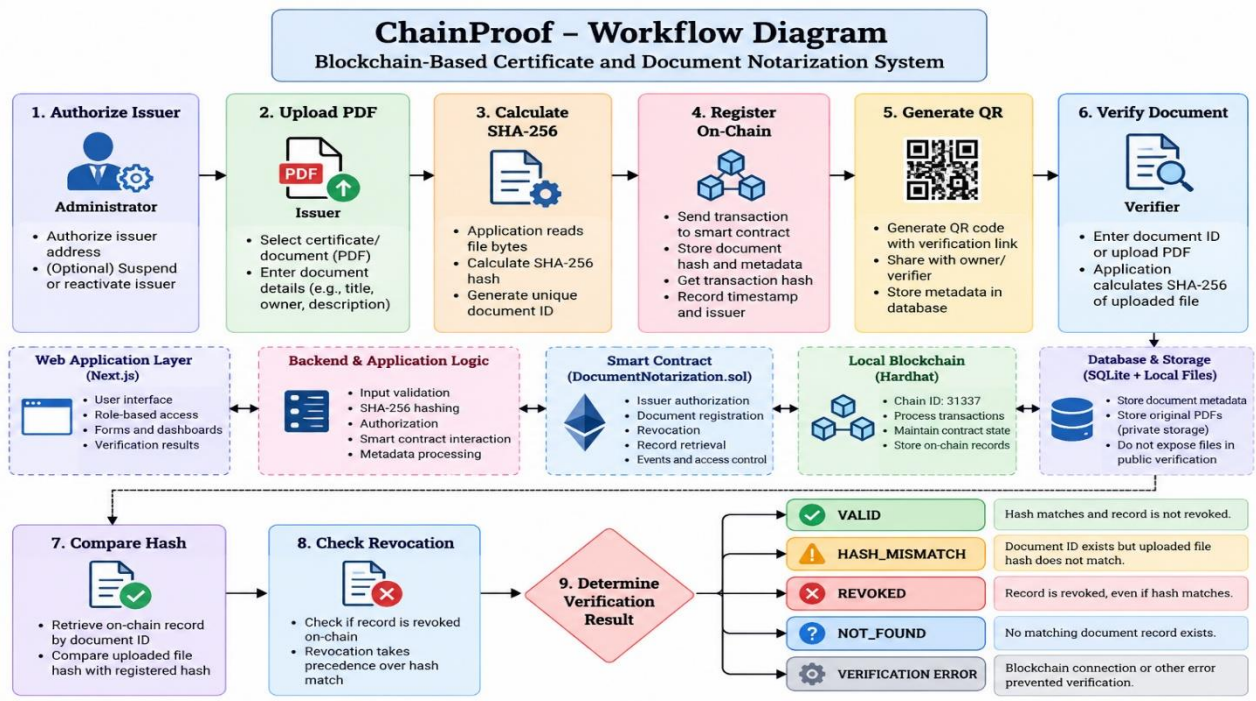
The application must not report a document as valid merely because the database contains a record or because the blockchain connection failed.

4.10 Use Case Workflow

The registration and verification process consists of the following steps:

1. The administrator authorizes an issuer.
2. The issuer uploads a PDF.
3. The application validates the file and calculates its SHA-256 hash.
4. The issuer submits the registration request.
5. The smart contract validates the issuer and records the document information.
6. The application records supporting metadata after transaction confirmation.
7. The system displays the document ID, hash, and transaction details.
8. A QR code links to the verification page.
9. The verifier uploads the original document or enters its ID.
10. The application checks the registered blockchain record.
11. The system displays the correct verification status.
12. If authorized, a user revokes the document and subsequent verification reports the revoked status.

Figure 4.2: Workflow of ChainProof



5. IMPLEMENTATION

5.1 Setting Up the Blockchain Environment

ChainProof is implemented using Solidity, Hardhat, and ethers.js for local blockchain development and interaction.

The smart contract is compiled and deployed to the local Hardhat network. The deployed contract address is:

0x5FbDB2315678afecb367f032d93F642f64180aa3

Network: Local Hardhat

Chain ID: **31337**

This address applies to the reported local deployment. A new deployment may produce a different address, and restarting the local blockchain resets its state.

The local setup allows the application to execute transactions and read contract state without requiring a public blockchain account or real cryptocurrency.

5.2 Defining the Smart Contract

5.2.1 DocumentNotarization Contract

The DocumentNotarization.sol contract manages issuer authorization and document records.

A document record is associated with its unique identifier and SHA-256 hash, together with relevant issuer, timestamp, and status information.

The contract ensures that registration is performed only by an authorized issuer. It also supports revocation and retrieval of registered records.

The main purpose of the contract is to enforce the rules governing the blockchain record rather than to store the complete document.

5.2.2 Issuer Authorization

Before registering a document, an issuer must be authorized by the administrator.

This restriction prevents an arbitrary blockchain account from registering documents as an approved issuer. The contract also supports suspension or reactivation according to the implemented authorization rules.

The application checks permissions before carrying out protected operations, but the smart contract independently enforces authorization for blockchain transactions.

5.2.3 Registering a Document

The issuer uploads a PDF and enters its required metadata.

The backend validates the uploaded file and calculates its SHA-256 hash. It then submits the hash and document identifier to the smart contract through a transaction.

After successful transaction confirmation, the application records the supporting metadata and presents the registration details.

The document is not considered successfully registered merely because a user submitted the form. The transaction must succeed.

5.2.4 Hash-Based Verification

The verifier uploads a file or enters a document identifier.

When a file is provided, the backend calculates the SHA-256 digest from the exact uploaded bytes. The application retrieves the registered record and compares the calculated hash with the on-chain hash.

If the hash matches and the record is not revoked, the application reports VALID. If the hashes differ, it reports HASH_MISMATCH.

A file that has been resaved may produce a different hash because the underlying bytes can change even when the visible content appears similar.

5.2.5 Document Revocation

The application allows a permitted administrator or issuer to revoke a document according to the smart contract's authorization rules.

The revocation operation is recorded through a blockchain transaction. During subsequent verification, revocation takes precedence over a matching hash.

This prevents a revoked record from being displayed as valid solely because the uploaded document remains unchanged.

5.2.6 Document Retrieval and QR Generation

The application supports lookup by document ID and QR-code generation.

The QR code contains a link to the corresponding verification page. It does not contain the original PDF or a direct public link to private document storage.

The verifier can open the page and, where necessary, upload the actual document to perform the hash comparison.

5.3 Frontend and Backend Integration

The frontend uses Next.js, React, TypeScript, and Tailwind CSS. It provides registration forms, dashboards, document details, issuer-management functions, and verification results.

Backend logic handles input validation, file hashing, authorization, metadata processing, and interaction with the smart contract.

The application uses SQLite through Prisma to store supporting metadata. Original PDFs are retained in private local storage rather than exposed as public static files.

Signed, short-lived role sessions protect application routes. Sensitive information such as original filenames and storage references is omitted from the relevant public dashboard and metadata responses.

5.4 Application Workflow

The working prototype demonstrates the following scenarios:

Scenario 1: Registering a Document
An authorized issuer selects a PDF and submits the registration request. The application calculates the file hash and submits the registration transaction. After confirmation, the application displays the record details.

Scenario 2: Verifying the Original Document
The verifier uploads the exact registered file. If the computed hash matches the registered hash and the record has not been revoked, the system returns VALID.

Scenario 3: Detecting a Modified Document
The verifier uploads a modified copy of a registered PDF. The computed hash differs from the registered hash, and the system returns HASH_MISMATCH.

Scenario 4: Revoking a Document
An authorized user revokes a document through a contract transaction. When the same original file is checked again, the system returns REVOKED.

Scenario 5: Searching for an Unknown Document
The verifier enters an identifier that has no corresponding registered record. The application returns NOT_FOUND.

Scenario 6: Rejecting Unauthorized Operations
An unauthorized issuer attempts to register a document or access protected functionality. The operation is rejected according to the application and contract authorization rules.

Scenario 7: Handling Blockchain Failure
If the application cannot obtain the required blockchain state, it displays a verification error rather than incorrectly declaring the document valid.

5.5 Testing and Verification

The prototype was tested using contract-level tests and application-level tests, followed by local smoke testing.

The reported pre-submission checklist contains the following results:

Test or validation	Reported result
Smart-contract tests	5 passed
Application tests	6 passed
Lint checks	Passed with no warnings or errors
Prisma validation	Passed
Production build	Passed
Registration and verification smoke tests	Passed
Modified-file detection	Passed
Revocation handling	Passed
Unknown document lookup	Passed
Role restrictions and metadata redaction	Passed
Direct local-chain check	Chain ID 31337 confirmed

The smoke tests cover registration, exact-file verification, modified-file detection, revocation, unknown document IDs, role restrictions, metadata redaction, and private-file URL checks.

These are reported implementation results and should be updated if the project is changed or the tests are rerun with different outcomes.

5.6 Limitations of the Implementation

The current prototype uses a local Hardhat blockchain rather than a public production network. Its local authentication arrangement is intended for demonstration and is not a complete production identity system.

MetaMask integration and optional Polygon Amoy deployment are not implemented in the reported version.

The system also depends on the availability of its local application, database, file storage, and blockchain process. Restarting the Hardhat node resets its blockchain state, requiring redeployment and reauthorization.

A production deployment would require additional security review, reliable backup and recovery, stronger identity management, operational monitoring, and a carefully managed blockchain deployment.

6. BENEFITS

6.1 Improved Document Integrity Verification

SHA-256 hashing provides a consistent method of comparing a submitted file with its registered version. A modified file can be detected when its digest no longer matches the registered digest.

6.2 Traceable Registration

The smart contract records registration transactions that can be referenced using transaction hashes. This improves traceability compared with a system that relies exclusively on editable application records.

6.3 Controlled Issuer Authorization

Issuer authorization restricts who can register document records through the contract. This reduces the risk of arbitrary accounts registering documents under the system's authorized-issuer workflow.

6.4 Efficient Verification

A verifier can check a document through the application without necessarily contacting the issuer for every individual verification request.

The system compares the uploaded file with the registered reference and retrieves the record's status from the blockchain.

6.5 Detection of Modified Files

A change to the document bytes normally results in a different SHA-256 digest. The application can therefore identify files that do not match the registered version.

6.6 Revocation Support

The revocation mechanism allows an authorized participant to mark a record as revoked. The verification workflow checks the revocation state before returning a valid result.

6.7 Reduced On-Chain Storage

The original PDF remains off-chain, while the contract records only the information required for registration and verification. This avoids placing complete documents on the blockchain.

6.8 Better Auditability

Registration and revocation transactions provide a history of important operations. These transaction references support review of the actions performed in the prototype.

6.9 QR-Based Access

QR codes provide a convenient way to open the appropriate verification page. They reduce the need to manually enter a lengthy document identifier or search for the verification interface.

6.10 Educational Value

The prototype demonstrates the integration of Solidity, smart contracts, cryptographic hashing, blockchain transactions, web development, databases, access control, and automated testing in a single application.

It also illustrates the difference between blockchain state and off-chain application data.

7. CHALLENGES

7.1 Public-Network Deployment

The current application uses a local Hardhat network. Moving to a public network requires suitable RPC configuration, contract deployment, transaction funding, network validation, and deployment testing.

7.2 Transaction Confirmation and Failure

Blockchain transactions can fail because of authorization errors, incorrect configuration, insufficient funds on a public network, or connectivity problems. The application must distinguish a confirmed transaction from a submitted or failed transaction.

7.3 File Storage Security

The original document is stored off-chain. Its security depends on appropriate access controls, safe storage paths, filesystem permissions, backup practices, and protection against unauthorized access.

7.4 Privacy Considerations

Blockchain records may be visible to participants who can inspect the relevant network. Personal information and complete documents should not be placed on a public blockchain without a strong justification.
ChainProof limits on-chain data to information required for its integrity and status workflow.

7.5 Issuer Trust

A matching hash establishes consistency with a registered file but does not prove that the original document was genuine. The system relies on appropriate issuer authorization and trustworthy initial registration.

7.6 Authentication Limitations

The local prototype uses signed, short-lived role sessions. A production implementation would require stronger identity verification, secure account lifecycle management, key management, and an appropriate administrative authorization process.

7.7 Network Availability

Verification requires access to the relevant blockchain state. If the local node or RPC connection is unavailable, the application cannot reliably complete the on-chain check and must

report an error.

7.8 File Hash Sensitivity

Hash-based verification compares exact file bytes. Saving or modifying a PDF can change its digest even when its visible content appears unchanged. Users must understand this behavior to avoid interpreting every mismatch as proof of intentional fraud.

7.9 Scalability and Operational Costs

A public blockchain may introduce transaction fees, confirmation delays, and throughput limitations. A large-scale system would need to assess the selected network, transaction volume, storage requirements, and operational costs.

7.10 Production Readiness

The prototype demonstrates core functionality but is not a complete production deployment. Security audits, deployment monitoring, robust backup and recovery, privacy review, and production-grade authentication would be necessary before real-world adoption.

8. CONCLUSION

ChainProof demonstrates a blockchain-based approach to certificate and document integrity verification. The application combines SHA-256 hashing, Solidity smart contracts, a local Hardhat blockchain, a web interface, and off-chain storage to provide registration, verification, issuer authorization, and revocation functionality.

The prototype records document hashes and relevant status information through blockchain transactions while keeping the original PDFs in private local storage. During verification, the application compares the uploaded file's digest with the registered hash and checks the revocation state. This allows the system to distinguish matching files, modified files, revoked records, unknown documents, and verification failures.

The implementation also demonstrates the importance of access control, safe handling of files, accurate transaction status reporting, and separation between on-chain and off-chain data.

The reported tests cover the principal registration and verification workflows, including modified-file detection, revocation, unknown records, and protected routes. The current deployment is a local development prototype and should not be confused with a public production blockchain deployment.

8.1 Future Enhancements

Future improvements may include:

1. Deployment to a suitable public testnet and, after appropriate review, a production network.
2. MetaMask integration for direct wallet-based transaction signing.
3. Stronger production authentication and identity verification.
4. Issuer verification through institutional onboarding.
5. Secure cloud or decentralized storage with appropriate privacy controls.
6. Improved monitoring, backup, and recovery.
7. More extensive automated testing and independent smart-contract security auditing.
8. Support for additional document formats.
9. Better institutional integration through authorized APIs.
10. Performance and scalability evaluation under larger workloads.

These improvements would help move the project from an academic prototype toward a more robust document verification service.

9. SDGs ADDRESSED

ChainProof has potential relevance to the following United Nations Sustainable Development Goals (SDGs).

SDG 4: Quality Education

Reliable certificate verification can help educational institutions, employers, and students check whether submitted files match previously registered academic records. This may support trustworthy academic credential handling.

However, blockchain hashing alone does not establish that the information in a certificate is true. Appropriate issuing institutions and authorization procedures remain necessary.

10. REFERENCES

[1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.
Available: https://bitcoin.org/bitcoin.pdf

[2] Ethereum Foundation, “Introduction to Smart Contracts.”
Available: https://ethereum.org/en/developers/docs/smart-contracts/

[3] National Institute of Standards and Technology (NIST), “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015. Available: https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[4] Solidity Documentation, “Solidity Programming Language.”
Available: https://docs.soliditylang.org/

[5] Hardhat, “Hardhat Documentation.”
Available: https://hardhat.org/docs

[6] ethers.js, “Documentation.”
Available: https://docs.ethers.org/

[7] Next.js, “Next.js Documentation.”
Available: https://nextjs.org/docs

[8] Prisma, “Prisma ORM Documentation.”
Available: https://www.prisma.io/docs

[9] United Nations, “Sustainable Development Goals.”
Available: https://sdgs.un.org/goals

11. APPENDIX A

Git Link :
https://github.com/lsv-7/BCT

